

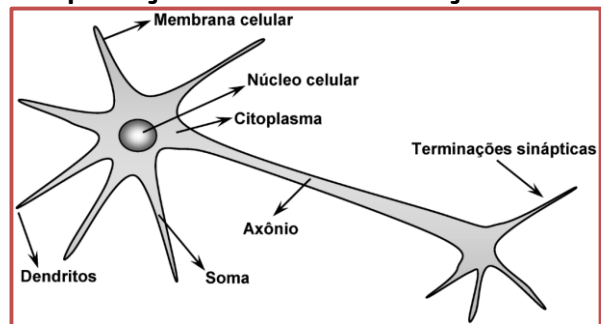


- Necessidade de máquinas com mecanismos autônomos, que sejam dotados de “inteligência”, a fim de tomarem decisões ou auxiliarem nestas mesmas.
- Várias aplicações práticas
- Medicina
- Química
- Biologia
- Finanças e Economia
- Classificação de Padrões
- Farmácia
- Engenharia
- Indústria de Alimentos
- Etc...



## Conceitos Iniciais

- São modelos computacionais inspirados no sistema nervoso de seres vivos (Computação Bio-inspirada).
- Possuem capacidade de aquisição e manutenção do conhecimento.
- Neurônio Biológico



## Principais Características

- Adaptação por experiência
- Capacidade de aprendizado
- Habilidade de generalização
- Organização de dados
- Tolerância a falhas
- Armazenamento distribuído
- Facilidade de prototipagem



## Resumo Histórico

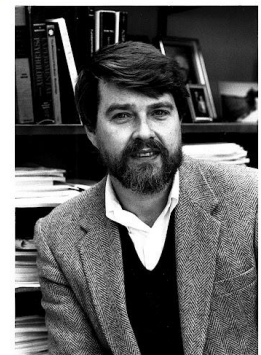
- McCulloch & Pitts (1943)
  - 1º modelamento matemático inspirado em neurônio
- Hebb (1949)
  - 1ª regra de aprendizado (Regra de Hebb)
- Frank Rosenblatt (1957-58)
  - 1º neurocomputador *Mark I - Perceptron*
- Widrow e Hoff (1960)
  - Redes *Adaline* (ADAptive LINEar Element)
  - Regra Delta
- Revés em 1969 – Minsk & Papert



Donald Hebb

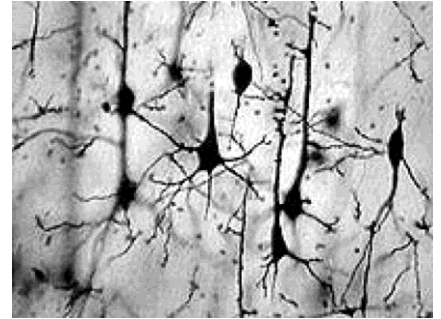
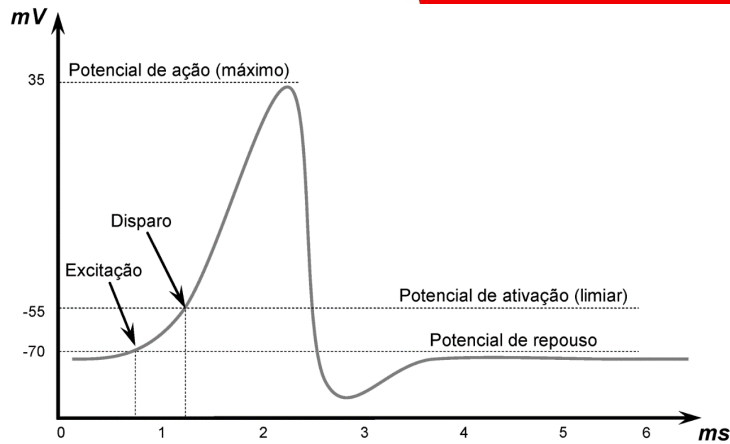
## Resumo Histórico

- Longo período de escassez...
- Grossberg (1980)
  - Rede ART (Adaptive Resonance Theory)
- Kohonen (1982)
  - Mapas auto-organizáveis
- Hopfield (1982)
  - Redes recorrentes
- Rumelhart, Hinton e Williams (1986)
  - *Parallel distributed processing*
  - Algoritmo de ajustes de pesos em multicamadas
  - *Backpropagation*

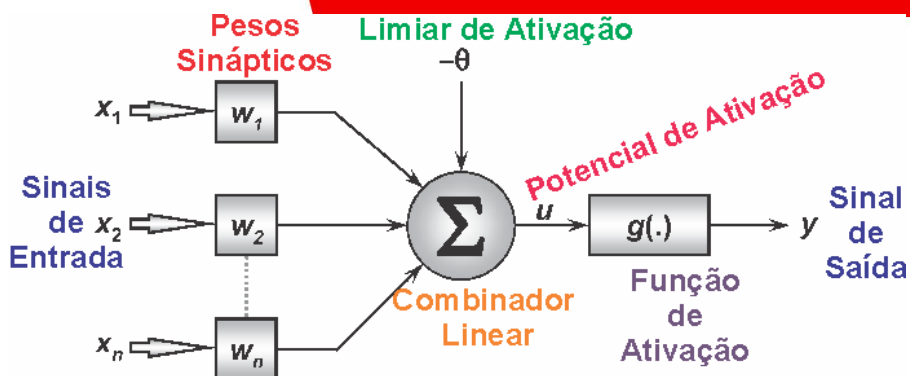


Rumelhart

# Funcionamento Cerebral



# Neurônio Artificial

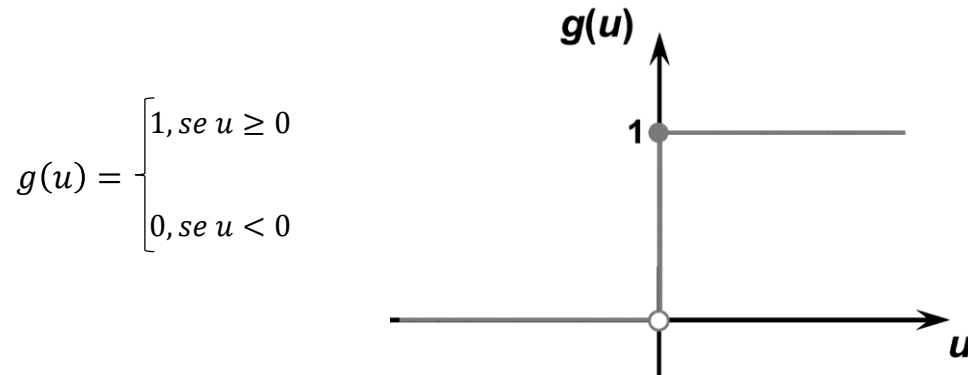


- Funções de Ativação são divididas em 2 grupos
  - Parcialmente diferenciáveis
  - Totalmente diferenciáveis



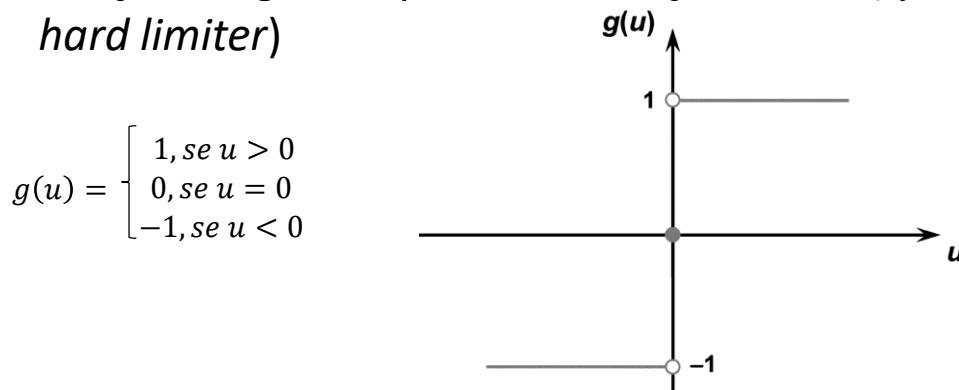
## Funções Parcialmente Diferenciáveis

- Função Degrau (*heavyside / hard limiter*)



## Funções Parcialmente Diferenciáveis

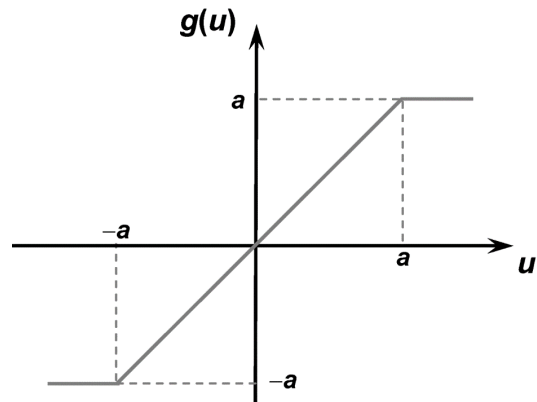
- Função Degrau Bipolar ou Função Sinal (*symmetric hard limiter*)



# Funções Parcialmente Diferenciáveis

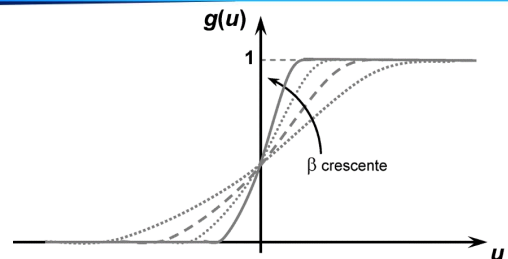
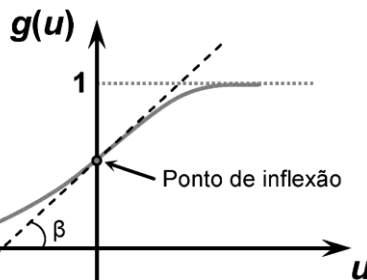
- Função Rampa Simétrica

$$g(u) = \begin{cases} a, & \text{se } u > a \\ u, & \text{se } -a \leq u \leq a \\ -a, & \text{se } u < -a \end{cases}$$



# Funções Totalmente Diferenciáveis

- Função Logística

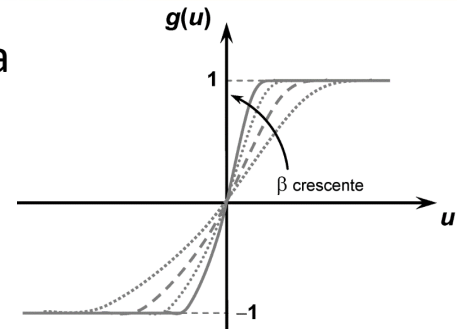
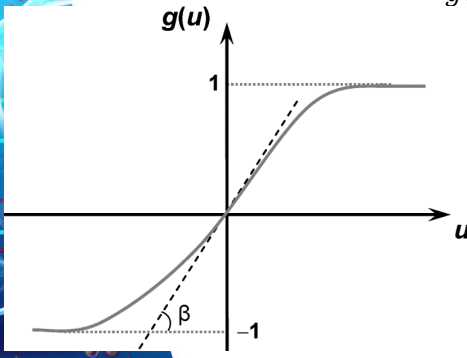


$$g(u) = \frac{1}{1 + e^{-\beta u}}$$

## Funções Totalmente Diferenciáveis

- Função Tangente Hiperbólica

$$g(u) = \frac{1 - e^{-\beta u}}{1 + e^{-\beta u}}$$

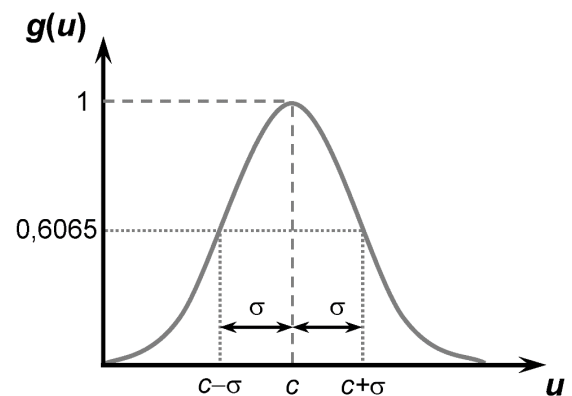


OBS: Tanto a função logística como a tangente hiperbólica são denominadas **sigmoidais**

## Funções Totalmente Diferenciáveis

- Função Gaussiana

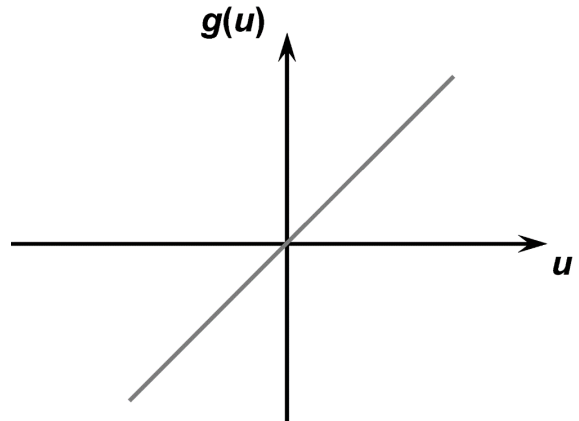
$$g(u) = e^{-\frac{(u-c)^2}{2\sigma^2}}$$



## Funções Totalmente Diferenciáveis

- Função Linear

$$g(u) = u$$



## Redes Neurais Artificiais

- **Arquitetura de uma RNA** → define a forma como seus diversos neurônios estão arranjados, ou dispostos, uns em relação aos outros.
- **Topologia de uma RNA** → São as diferentes formas de composições estruturais que a RNA pode assumir.
- **Treinamento de uma RNA** → Aplicação de um conjunto de passos ordenados com o intuito de ajustar os pesos e limiares de seus neurônios.



## Divisão de uma RNA

- Basicamente uma RNA é dividida em 3 partes:
  - Camada de Entrada
    - Recebe as informações (dados)
  - Camadas Escondidas, intermediária, ocultas ou invisíveis
  - Camada de Saída
    - Produção dos resultados finais

## Divisão de uma RNA

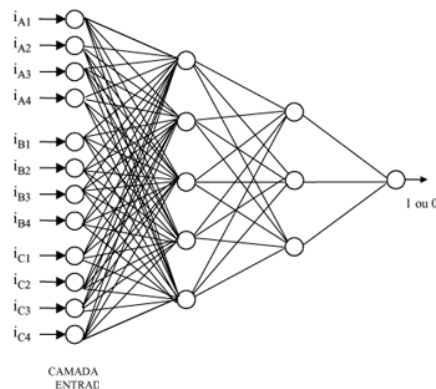
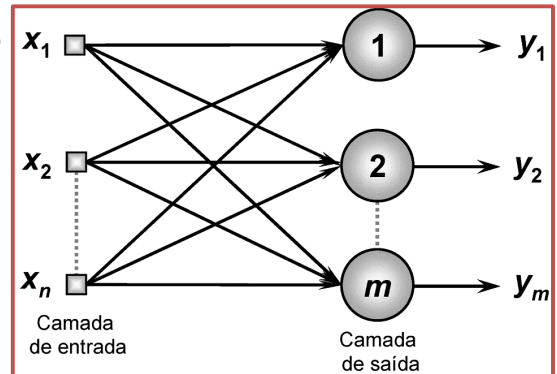


Figura 7: Representação das entradas e saídas das RNAs

## Arquitetura *feedforward* de camada simples

- Apenas 1 camada de entrada e 1 camada de neurônios que é a própria de saída

— Exº. Perceptron e Adaline

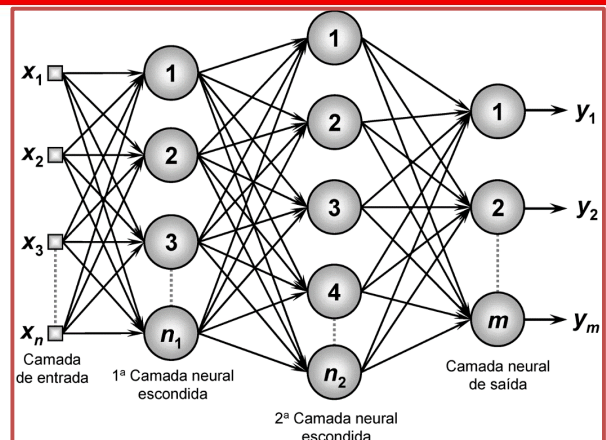


## Arquitetura *feedforward* de camadas múltiplas

- Além da camada de entrada e saída possui uma ou mais camadas escondidas

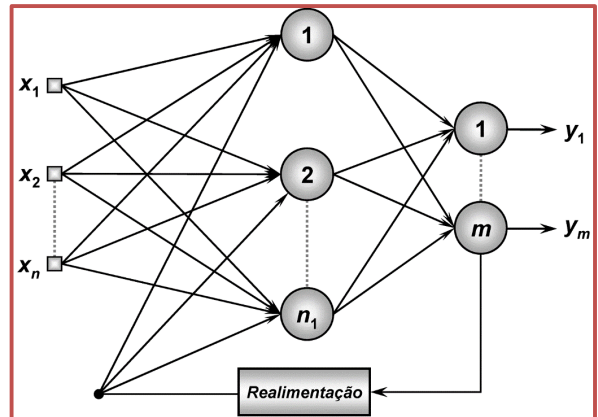
— Exº. MLP e

RBF (*Radial Basis Function*)



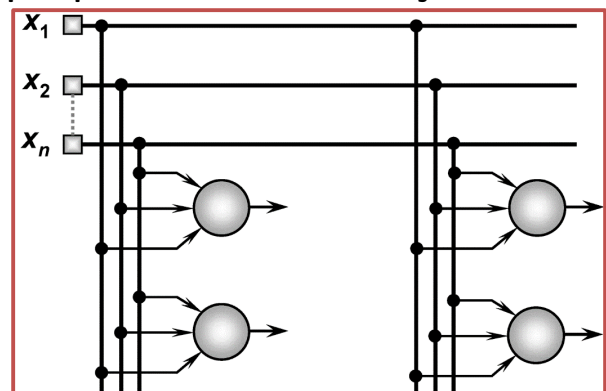
## Arquitetura recorrente ou realimentada

- Saídas dos neurônios são realimentadas como sinais de entrada para outros neurônios
  - Exº. MLP com realimentação e Hopfield



## Arquitetura em estrutura reticulada

- A principal característica está na disposição espacial dos neurônios visando propósitos de extração de características
  - Exº. Rede de Kohonen



## Processos de Treinamento

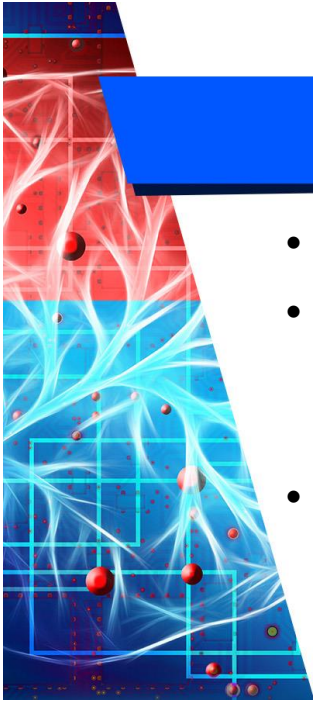


- Uma RNA possui a capacidade de adaptar-se de acordo com dados de entrada pré-existent (dados de treinamento) que realizam a modificação dos pesos sinápticos e limiares de seus neurônios.
- Após esta etapa a RNA é capaz de generalizar soluções a partir de quaisquer sinais inseridos.

## Treinamento Supervisionado

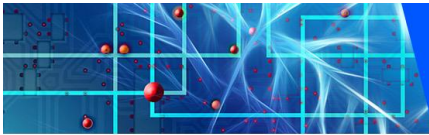


- Tem-se disponível para cada amostra de entrada a respectiva saída.
- Após o treinamento é efetuado um teste com estes dados.
  - Treinar novamente enquanto for necessário
  - Rede Treinada



## Treinamento Não-supervisionado

- Inexistência dos dados de saída
- A rede se auto-organiza em relação a particularidades entre os elementos da amostra formando subconjuntos (*Clusters*)
- Pode-se especificar a quantidade dos *clusters*



## Treinamento com Reforço



- Variante do treinamento supervisionado.
- Treinamento dentro de um processo de tentativa e erro
  - Saída satisfatória ou não?
- Pesos sinápticos são graduados visando “reforçar” a condição do sistema



## Aprendizagem

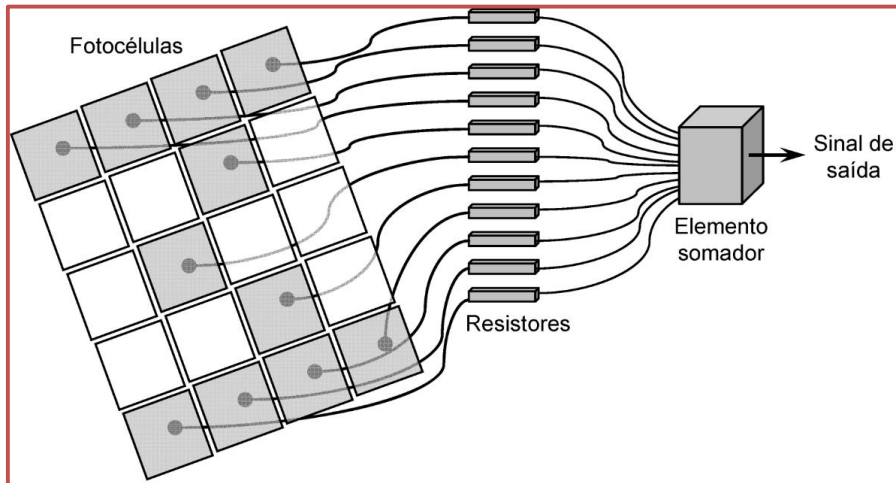
- Usando Lotes de Padrão (*off-line* ou *batch*)
  - As modificações dos pesos e limiares só são efetivados após a apresentação de todos o conjunto de treinamento
- Usando Padrão-Por-Padrão (*on-line*)
  - Os ajustes são efetivados a cada amostra do conjunto de treinamento



## Rede Perceptron

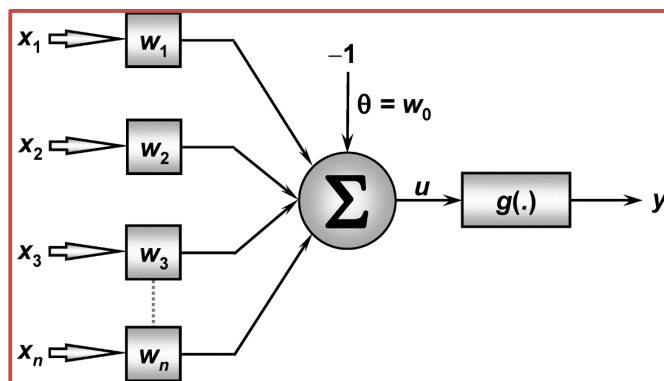
- Idealizada por Rosenblatt (1958)
- Forma mais simples de uma rede neural artificial
  - Inspirado na retina humana
- Aplicados na identificação de padrões geométricos

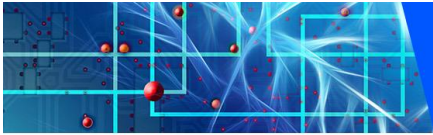
# Rede Perceptron



## Estrutura Perceptron

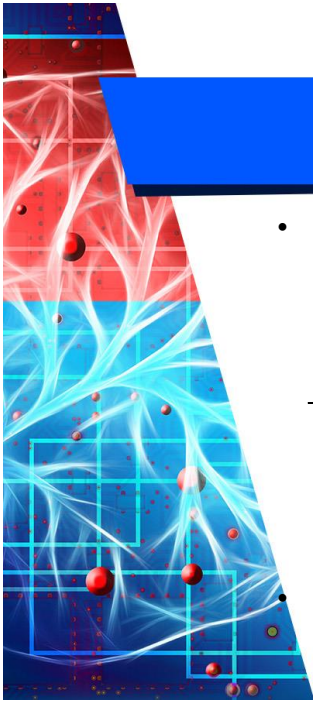
- Apenas 1 camada com apenas 1 neurônio artificial
  - Rede *feedforward* de camada única





## Princípio de Funcionamento

- Cada entrada ( $x_i$ ) é ponderada por seu respectivo peso ( $w_i$ )
- Forma-se o argumento  $u$  para a função de ativação com a soma dos pesos unidos com o limiar do neurônio.
- Matematicamente:
 
$$\begin{cases} u = \sum_{i=1}^n w_i \cdot x_i - \theta \\ y = g(u) \end{cases}$$
- Normalmente a função de ativação é:
  - Degrau
  - Degrau bipolar
- Treinamento supervisionado e aprendizado pela regra de Hebb



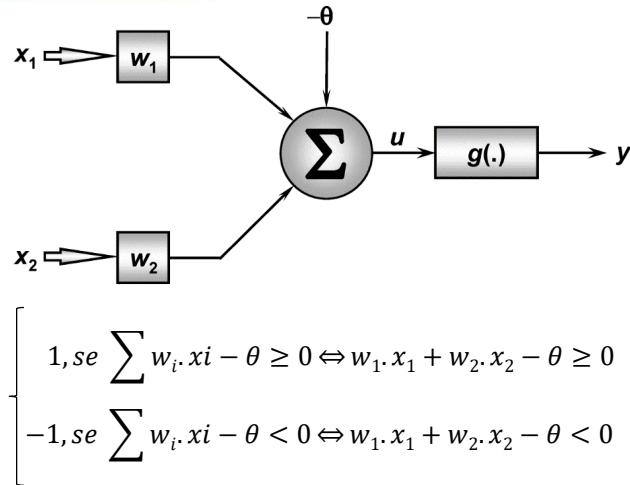
## Análise Matemática do Perceptron

- Vamos adotar uma rede com apenas 2 (duas) entradas

$$\begin{cases} 1, \text{ se } \sum w_i \cdot x_i - \theta \geq 0 \Leftrightarrow w_1 \cdot x_1 + w_2 \cdot x_2 - \theta \geq 0 \\ -1, \text{ se } \sum w_i \cdot x_i - \theta < 0 \Leftrightarrow w_1 \cdot x_1 + w_2 \cdot x_2 - \theta < 0 \end{cases}$$

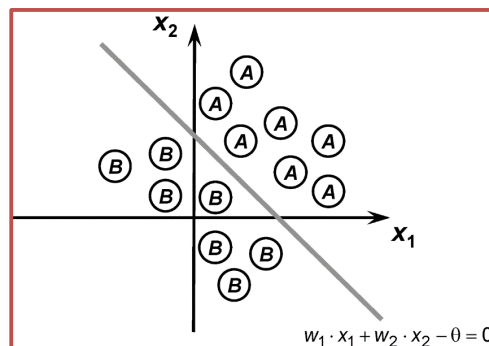
- As desigualdades são do primeiro grau e a reta gerada desta equação forma a **fronteira de decisão**

## Análise Matemática do Perceptron



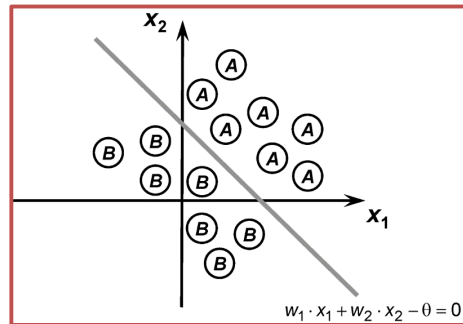
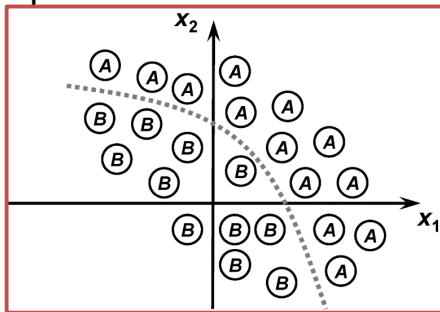
## Fronteira de Separação

- Perceptron com duas entradas
  - Classes linearmente separáveis



## Teorema de Convergência do Perceptron

- Um Perceptron de camada simples (única) pode ser utilizado para classificação de padrões se estes forem linearmente separáveis.



## Treinamento do Perceptron

- Regra de aprendizado de Hebb
- Compara-se a saída da rede com a saída desejada
  - IGUAIS → Pesos e limiares serão incrementados proporcionalmente aos valores dos sinais de entrada
  - DIFERENTES → Pesos e limiares serão decrementados proporcionalmente aos valores dos sinais de entrada
- Em termos matemáticos:

$$\begin{cases} w_i^{atual} = w_i^{anterior} + \eta \cdot (d^{(k)} - y) \cdot x^{(k)} \\ \theta_i^{atual} = \theta_i^{anterior} + \eta \cdot (d^{(k)} - y) \cdot x^{(k)} \end{cases}$$

- Entretanto em termos computacionais, torna-se mais conveniente tratarmos os elementos como vetoriais



# Treinamento do Perceptron

- O limiar pode ser inserido no vetor dos pesos sinápticos

$$\mathbf{w}^{atual} = \mathbf{w}^{anterior} + \eta \cdot (d^{(k)} - y) \cdot \mathbf{x}^{(k)}$$

ou

$$\mathbf{w}^{atual} \leftarrow \mathbf{w}^{anterior} + \eta \cdot (d^{(k)} - y) \cdot \mathbf{x}^{(k)}$$

- Onde:

$\mathbf{w} = [\theta \ w_1 \ w_2 \ \dots \ w_n]^T$  É o vetor contendo o limiar e os pesos

$\mathbf{x}^{(k)} = [-1 \ x_1^{(k)} \ x_2^{(k)} \ \dots \ x_n^{(k)}]^T$  É a  $k$ -ésima amostra de treinamento

$d^{(k)}$  É o valor desejado para a  $k$ -ésima amostra de treinamento

$y$  É o valor da saída produzida pelo *Perceptron*

$\eta$  É a constante que define a taxa de aprendizagem da rede

Esta constante ( $\eta$ ) exprime o quão rápido o processo de treinamento está sendo conduzido rumo à convergência (estabilização). Geralmente  $0 < \eta < 1$

# Implementação Computacional

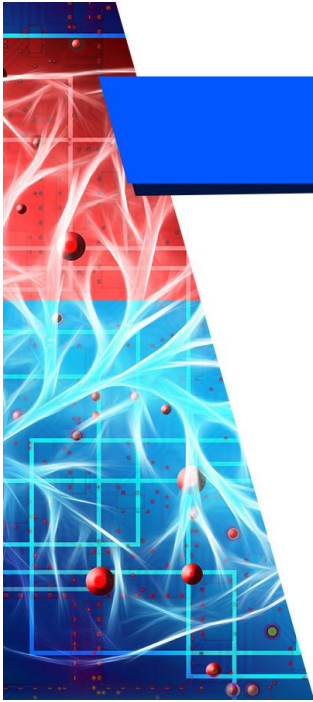
## Início {Algoritmo *Perceptron* – Fase de Treinamento}

```

<1> Obter o conjunto de amostras de treinamento  $\{\mathbf{x}^{(k)}\}$ ;
<2> Associar a saída desejada  $\{d^{(k)}\}$  para cada amostra obtida;
<3> Iniciar o vetor  $\mathbf{w}$  com valores aleatórios pequenos;
<4> Especificar a taxa de aprendizagem  $\{\eta\}$ ;
<5> Iniciar o contador de número de épocas  $\{\acute{e}poca \leftarrow 0\}$ ;
<6> Repetir as instruções:
    {
        <6.1> erro  $\leftarrow$  "inexiste";
        <6.2> Para todas as amostras de treinamento  $\{\mathbf{x}^{(k)}, d^{(k)}\}$ , fazer:
            {
                <6.2.1>  $u \leftarrow \mathbf{w}^T \cdot \mathbf{x}^{(k)}$ ;
                <6.2.2>  $y \leftarrow \text{sinal}(u)$ ;
                <6.2.3> Se  $y \neq d^{(k)}$ 
                    {
                        <6.2.3.1> Então  $\begin{cases} \mathbf{w} \leftarrow \mathbf{w} + \eta \cdot (d^{(k)} - y) \cdot \mathbf{x}^{(k)} \\ \text{erro} \leftarrow \text{"existe"} \end{cases}$ 
                    }
            }
        <6.3>  $\acute{e}poca \leftarrow \acute{e}poca + 1$ ;
        Até que: erro  $\leftarrow$  "inexiste"
    }

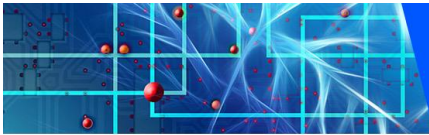
```

## Fim {Algoritmo *Perceptron* – Fase de Treinamento}



## Operação da Perceptron

- Verifica-se a existência de uma variável “época” que contará quantas vezes a rede precisará ler todas as amostras para ser considerada treinada.
- Uma vez que a rede esteja treinada ela está apta para ser operada gerando novos resultados para dados ainda não lidos. (identificações de padrão)



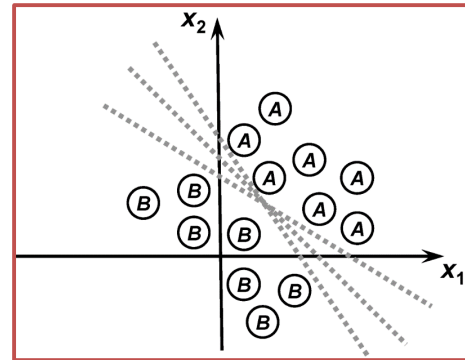
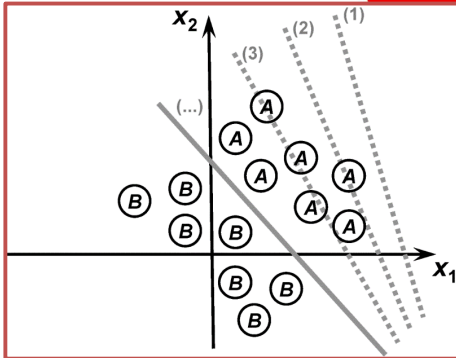
## Operação da Perceptron

### Início {Algoritmo *Perceptron* – Fase de Operação}

- <1> Obter uma amostra a ser classificada  $\{ \mathbf{x} \}$ ;
- <2> Utilizar o vetor  $\mathbf{w}$  ajustado durante o treinamento;
- <3> Executar as seguintes instruções:
  - <3.1>  $u \leftarrow \mathbf{w}^T \cdot \mathbf{x}$ ;
  - <3.2>  $y \leftarrow \text{sinal}(u)$ ;
  - <3.3> Se  $y = -1$ 
    - <3.3.1> Então: amostra  $\mathbf{x} \in \{\text{Classe A}\}$
  - <3.4> Se  $y = 1$ 
    - <3.4.1> Então: amostra  $\mathbf{x} \in \{\text{Classe B}\}$

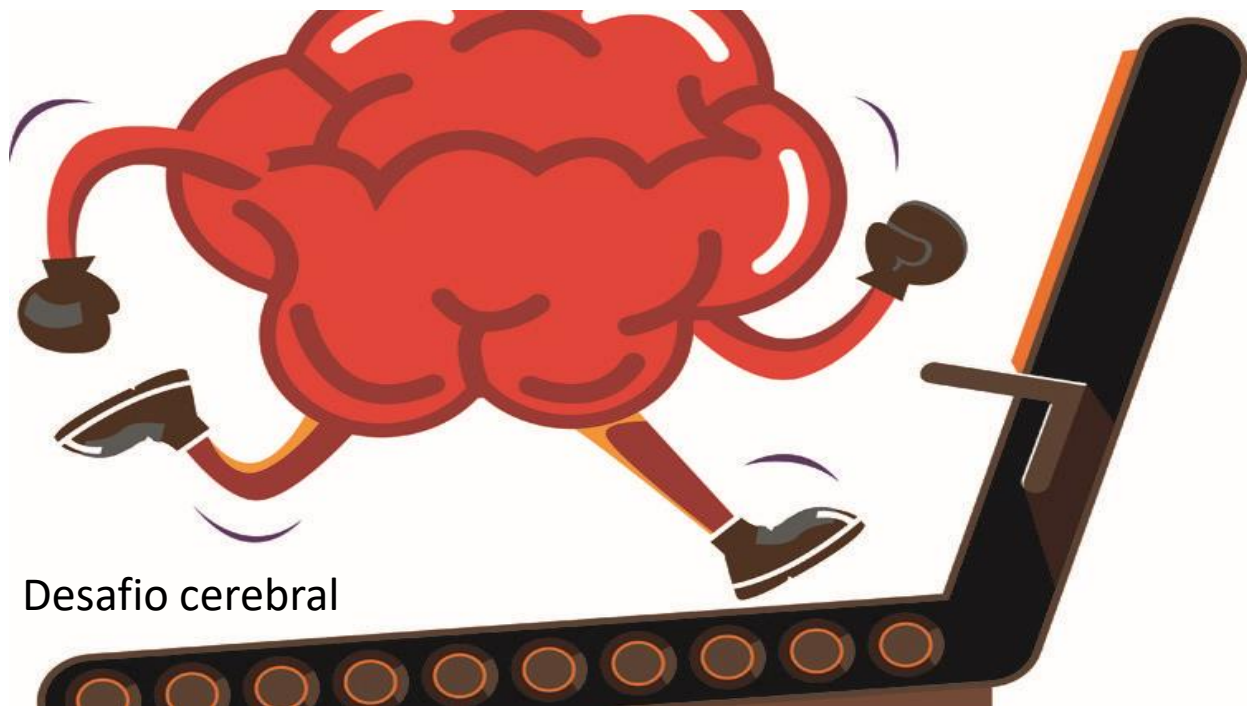
### Fim {Algoritmo *Perceptron* – Fase de Operação}

## Visualizando o Treinamento



## Aspectos importantes para o treinamento

- A rede divergirá se o problema não for linearmente separável
  - Limitar o número de épocas
- Se a faixa de separabilidade for muito estreita entre duas classes o processo ficará instável
  - Atribuir um valor pequeno para a taxa de aprendizado  $\eta$
- A quantidade de épocas varia de acordo com os valores iniciais dos pesos, da taxa de treinamento e da ordem das amostras
- Quanto mais próxima a superfície de decisão estiver da fronteira de separabilidade, menos épocas
- Normalização das entradas contribui para um melhor desempenho da rede



Desafio cerebral